DSA Assignment 1
CS F24 AFTERNOON
Expression Conversion and Evaluation

Objective

The objective of this assignment is to design and implement a system capable of processing arithmetic expressions. The program must accept an expression written in **infix notation**, convert it to **postfix notation**, and evaluate the resulting expression.

Problem Description

Implement a program that reads an arithmetic expression consisting of:

- Binary operators: `+`, `-`, `*`, `/`
- Grouping symbols: `()`, `[]`, {}
- Integer constants
- Variables with names that comply with the `C++` identifier rules

The program shall process the expression through several stages, as outlined below.

Required Functionality

Tokenization and Parsing

The program must first decompose the input expression into a sequence of tokens and construct an internal representation based on these tokens.

Conversion to Postfix Notation

The infix expression must be transformed into **postfix notation** (Reverse Polish Notation).

For instance, the expression:

$$a + b \times (c + d)$$

may be converted to:

$$a\ b\ c\ d\ +\ \times\ +$$

Operator precedence shall follow standard arithmetic rules:

$$*, /\quad >\quad +, -$$

All grouping symbols `()`, `[]`, and {} must be consistently handled during the conversion process.

Variable Value Acquisition

All variables present in the expression must be identified. The program should prompt the user to provide a numeric value for each variable.

Postfix Evaluation

The postfix expression must be evaluated using the prompted values. The program shall compute and output the resulting value.

Input

A single line containing an infix arithmetic expression.

Output

The program shall display:

- The postfix form of the expression
- The evaluated result

Important: - All prompts for variable values and intermediate messages must be written to `stderr`. - Only the final evaluated result must be written to `stdout`.

Exit Codes and Error Handling

To support automated testing, the program shall terminate with specific exit codes depending on the type of error encountered:

- **Exit code 0:** Successful execution; expression evaluated correctly.
- **Exit code 1:** Syntax error in the expression. The program shall print an appropriate error message to `stderr`.

- **Exit code 2:** Runtime error during evaluation. The program shall print an appropriate error message to `stderr`.
- **Exit code 3:** Logical error in the expression. The program shall print an appropriate error message to `stderr`.

Example

Input:

```
a + b * (c + 2)
```

Variable Queries (stderr):

```
Enter value for a: 3
Enter value for b: 5
Enter value for c: 2
```

Final Result (stdout):

```
a b c 2 + * +
23
```

Submission Requirements

Students shall submit:

- A GitHub repository link containing the program code, with at least three meaningful commits.
- A README file providing instructions for compilation and usage.